

Adaptive QoS with Reinforcement Learning

Overview

This project investigates whether a reinforcement-learning controller can dynamically select Quality of Service (QoS) configurations on a MikroTik router. Traffic is divided into three classes using Differentiated Services Code Point (DSCP) markings. The intended objective is to protect high- and medium-priority traffic while preserving as much low-priority throughput as possible when the offered load exceeds a 20 Mbps bottleneck.

The completed system reads live Queue Tree rates and cumulative drop counters from RouterOS, converts them into a discrete state, selects one of three safe, predefined queue profiles, applies the profile through the RouterOS REST API, calculates a reward, and updates a tabular Q-learning model. The experiment successfully protected priority traffic, but the final controller is a hybrid: explicit rules select a protective profile whenever priority traffic is active, while Q-learning makes decisions only when the high- and medium-priority queues are inactive. This distinction is central to the interpretation of the results.

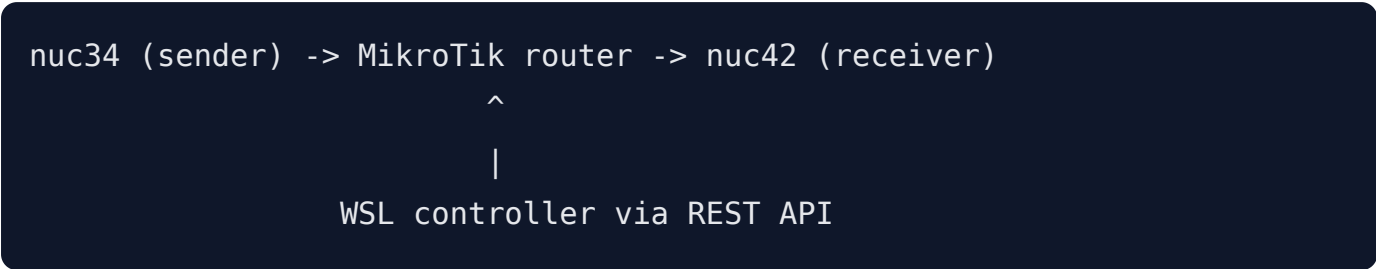
Approach and Environment

Hardware, software, and topology

Two Xubuntu 26.04 computers generate and receive experiment traffic through a MikroTik router running RouterOS 7.23.2. A separate Ubuntu environment in Windows Subsystem for Linux (WSL) runs the controller. It communicates with the router over the management network and is not in the experiment traffic path.

Component	Address	Role
nuc34	192.168.4.2	Generates UDP traffic
MikroTik router	management: 192.168.88.34	Classifies, queues, and forwards traffic
nuc42	192.168.5.2	Receives UDP traffic

Windows laptop with Ubuntu/WSL	management network	Runs monitoring and the controller
--------------------------------	--------------------	------------------------------------



All experiment traffic crosses a 20 Mbps Queue Tree parent on router interface `ether5` . RouterOS mangle rules classify DSCP 46, 26, and 0 packets into the `qos-high` , `qos-medium` , and `qos-low` child queues. The router is therefore both the controlled bottleneck and the component that enforces each action.

Traffic workloads

The traffic generator uses three simultaneous `iperf3` UDP streams. UDP was chosen because it maintains the requested offered load under congestion, making packet loss directly observable. DSCP 46 represents high-priority, latency-sensitive traffic; DSCP 26 represents medium-priority interactive traffic; and DSCP 0 represents low-priority bulk traffic.

The current targets in `scripts/start_iperf3_clients.sh` are:

Workload	High, DSCP 46	Medium, DSCP 26	Low, DSCP 0	Total offered
<code>open</code> / profile 0	0	0	20 Mbps	20 Mbps
<code>minor</code> / profile 1	0.5 Mbps	0.5 Mbps	20 Mbps	21 Mbps
<code>moderate</code> / profile 2	1 Mbps	1 Mbps	20 Mbps	22 Mbps
<code>major</code> / profile 3	2 Mbps	2 Mbps	20 Mbps	24 Mbps

A zero-rate stream is not launched. The current script runs each workload for 60 seconds. The archived result files used in this report came from earlier 300-second runs, as shown by their `iperf3` summary intervals. This does not change the bandwidth targets or the conclusions, but it is documented so the experiment duration is not misrepresented.

Rules of the RL environment

The controller repeats an observation-action-reward loop every five seconds:

1. Read each child queue's current rate and cumulative drop counter.
2. Convert the difference from the previous drop reading and the current rate into a discrete state.
3. Select and apply one predefined queue profile.
4. Wait five seconds, observe the next state, calculate the reward, and update the Q-table.

The environment runs until stopped by the operator. The first drop reading is treated as a baseline. If a RouterOS counter decreases, it is treated as a counter reset rather than as a negative number of drops.

State

Each queue receives one status value:

Status	Meaning
0	Idle or at most 10 Kbps
1	Active with no new drops
2	New drops and rate at or below 750 Kbps
3	New drops and rate at or below 1.5 Mbps
4	New drops and rate above 1.5 Mbps

The complete state is `(high_status, medium_status, low_status, current_profile)`. The final implementation uses Queue Tree drop deltas as its loss signal and queue rates as its throughput signal. It does not measure round-trip latency directly, so the final report does not claim that latency was part of the learned state or reward.

Actions

Actions are limited to complete, validated configurations. Each action changes only the `limit-at` committed rate of the child queues; the queue structure and maximum rates remain unchanged.

Action	qos-high	qos-medium	qos-low
--------	----------	------------	---------

balanced	1 Mbps	1 Mbps	1 Mbps
protected	1.25 Mbps	1.25 Mbps	0.25 Mbps
maximum_protection	1.5 Mbps	1.5 Mbps	0.1 Mbps

This small action space made the experiment safer: the controller could not construct an invalid or unreviewed router configuration.

Model and action policy

The model is tabular Q-learning with learning rate `alpha = 0.2`, discount factor `gamma = 0.9`, and a fixed exploration rate `epsilon = 0.2`. It uses the standard update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha * (\text{reward} + \gamma * \max_{a'} Q(s', a') - Q(s, a))$$

For states that are not covered by a guardrail, the controller uses an epsilon-greedy policy and retains the current action when its Q-value is within 1.0 of the best action. This stability margin reduces unnecessary switching.

The final policy also contains three deterministic guardrails:

- an idle state selects `balanced`;
- active high- or medium-priority traffic with status 1 or 2 selects `protected`; and
- high or medium status 3 or 4 selects `maximum_protection`.

Reward

For a non-idle interval, the reward begins at 25 when high-priority traffic has no new drops, or 10 otherwise. It then:

- adds 4 for no medium-priority drops, or subtracts 8 when they occur;
- adds 2 for each high/medium rate at or above 900 Kbps, with a proportional partial reward below that threshold;
- adds 1 when low-priority traffic absorbs drops;
- subtracts 0.5 for `maximum_protection`; and
- subtracts 0.5 when the selected profile differs from the current profile.

An entirely idle interval receives zero reward. The positive baseline was added because an earlier reward made every congested action negative, causing untried actions initialized at zero to appear artificially superior.

Implementation and Problem-Solving Process

The system was built in stages. `monitor.py` first established reliable REST reads of the Queue Tree and `ether5` transmit rate. `profile_controller.py` then added a restricted write path for the three predefined actions. `run_controller.py` joined those pieces using a static controller before the Q-learning controller was introduced. This isolated router integration errors from learning-policy errors.

One live write test returned HTTP 400 even though reads worked. RouterOS Queue Tree resource identifiers begin with `*`; URL-encoding that character as `%2A` made the identifier invalid. Leaving the leading `*` literal fixed the update. The three queues were then successfully changed to the `balanced` committed rates and restored.

The learning algorithm also went through several iterations:

1. A fixed epsilon caused continued profile changes after one action had a higher learned value. A 0.5 switching penalty and 1.0 stability margin were added to discourage changes with little expected benefit.
2. The first state representation encoded every active queue with new drops in the same category. Rate bands were added to distinguish small, moderate, and large active flows when drops occurred.
3. The first congestion reward was negative for every tried action. Because unseen actions began at zero, they looked preferable. A constant 20-point offset was added to non-idle rewards without changing the relative ordering of outcomes.
4. A decaying-epsilon experiment converged to `balanced` for every workload, with similar router performance across actions. Since the reward did not establish that stricter profiles were better, I added explicit protection guardrails for the demonstration and restored fixed epsilon to 0.2 for states outside those rules.

The last change produced the desired network behavior, but it also changed the scientific result: the protective decisions in the final multi-class runs were

programmed rules rather than a policy learned from experience.

Experiments and Results

Four archived 300-second runs applied progressively heavier offered loads. The controller began in `balanced`; the sender then launched the selected workload.

The `iperf3` receiver summaries measure delivered bitrate, jitter, and loss.

Controller summaries below count each printed five-second decision. A profile transition includes returning to `balanced` when traffic ends.

Workload	High result	Medium result	Low result	Controller behavior
Profile 0: 0/0/20 Mbps	not launched	not launched	19.4 Mbps, 0.047 ms jitter, 2.8% loss	76 decisions; 7 transitions; used all three actions
Profile 1: 0.5/0.5/20 Mbps	0.5 Mbps, 0.025 ms, 0% loss	0.5 Mbps, 0.015 ms, 0% loss	18.4 Mbps, 0.063 ms, 7.8% loss	changed to <code>protected</code> when traffic began and back to <code>balanced</code> when idle
Profile 2: 1/1/20 Mbps	1.0 Mbps, 0.013 ms, 0% loss	1.0 Mbps, 0.019 ms, 0% loss	17.4 Mbps, 0.131 ms, 13% loss	changed to <code>protected</code> when traffic began and back to <code>balanced</code> when idle
Profile 3: 2/2/20 Mbps	2.0 Mbps, 0.391 ms, 0% loss	2.0 Mbps, 0.212 ms, 0.0019% loss	15.4 Mbps, 0.251 ms, 23% loss	changed to <code>protected</code> when traffic began and back to <code>balanced</code> when idle

The results show the intended QoS outcome clearly. As offered traffic rose from 20 to 24 Mbps, delivered low-priority throughput fell from 19.4 to 15.4 Mbps and its loss rose from 2.8% to 23%. At the same time, the high- and medium-priority streams delivered essentially their full offered rates with zero loss. The router successfully placed congestion on the low-priority class.

The controller behavior requires a more cautious interpretation. In profile 0, only DSCP 0 traffic was active, so the deterministic priority guardrails did not apply. The controller moved among `balanced`, `protected`, and `maximum_protection` several times as epsilon-greedy exploration and learned values operated. In profiles 1 through 3, it made one meaningful transition from `balanced` to `protected` after priority traffic appeared, held that profile throughout the active workload, and returned to `balanced` after the traffic ended. This was consistent with the algorithm: status 1 for the active high and medium queues explicitly maps to `protected`. However, it was not evidence that Q-learning discovered that action, and the multi-class runs did not exercise `maximum_protection` because the priority queues did not report the drop statuses that trigger it.

The rising Q-values in the logs are not probabilities. With `gamma = 0.9`, a repeated reward near 30 has a theoretical continuing-return scale near $30 / (1 - 0.9) = 300$, so Q-values above 200 in these runs are reasonable.

Discussion and Reflection

I am not satisfied with the final controller even though I am satisfied with the network performance. I can see that it reads real router statistics, calculates states and rewards, and updates Q-values, but its decisions are not the kind of adaptive learned behavior I originally wanted. I had never used RouterOS Queue Tree QoS before this project and underestimated how much work the router would already do once its queue hierarchy, priorities, and committed rates were configured correctly. The router did most of the useful traffic protection, leaving the Q-learning controller with little measurable advantage to discover among the three actions.

The final guardrails made the demonstration stable, but they also meant that the controller did not make most multi-class decisions through Q-learning. The profile 0 log is evidence that the learning path was active, while profiles 1–3 are primarily evidence that the rule-selected profile and RouterOS queues performed well. Based on the implemented algorithm, those decisions were not wrong; they simply do not demonstrate the degree of learned control I intended.

There are also experimental limitations. The final state and reward use router queue rates and drop deltas, not direct latency measurements. The result set does not include equal-workload static runs for all three queue actions, so it

cannot establish that the hybrid controller outperformed a well-chosen static profile. Each workload was represented by one archived run, which is enough to demonstrate the mechanism but not enough for statistical confidence.

Conclusion and Future Work

The project produced a working end-to-end system and achieved its operational QoS goal: priority UDP traffic retained almost all offered throughput and nearly zero loss while the low-priority stream absorbed increasing congestion. It did not establish that Q-learning was responsible for that result. My main conclusion is that a strong built-in queueing policy can reduce both the need for and the learning signal available to an external RL controller.

More tuning and a stronger evaluation design are required, but I have run out of time to complete them while meeting the project deadline. The most useful next steps would be:

- remove the deterministic guardrails and train across many randomized traffic episodes with epsilon decay and a persisted Q-table;
- compare every learned policy against static `balanced`, `protected`, and `maximum_protection` baselines under identical workloads;
- add active latency or round-trip-time measurements to the state and reward;
- create actions that materially change scheduling behavior, not only `limit-at`, so actions produce a clearer measurable learning signal;
- repeat each experiment and report means, variation, convergence, cumulative reward, and switching rate; and
- separate training from evaluation so evaluation runs use no exploration.

Deliverables and References

- [Project source repository](#)
- [Project notebook](#)
- [Traffic-generation scripts](#)
- [Raw experiment results](#)
- [Demo video](#)
- [MikroTik RouterOS REST API documentation](#)

- [MikroTik Queue documentation](#)